Introduction to Machine Learning

Lecture 18: CNNs and Sequence Data

Ali Bereyhi

ali.bereyhi@utoronto.ca

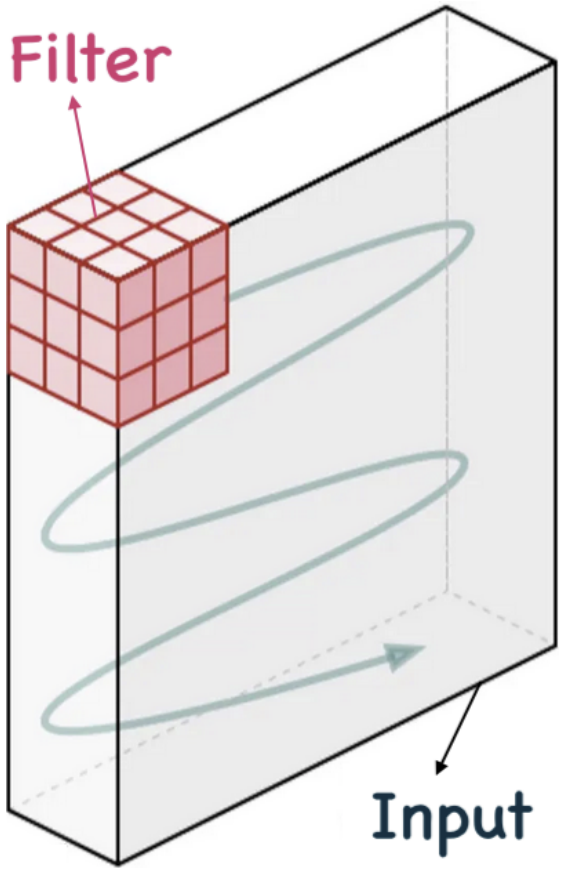
Department of Electrical and Computer Engineering

University of Toronto

Winter 2026

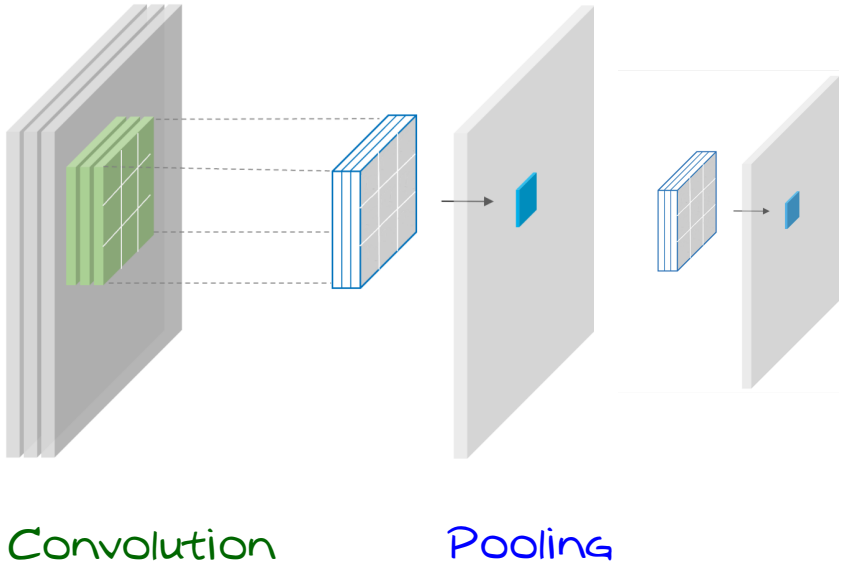
Recap: *Convolution*

In convolution, we slide a filter over the input sample



Recap: *Complex Convolutional Layer*

A convolutional unit is made as



- *The convolution performs less complex linear operation*
- *Pooling make the output smooth, i.e., less varying*

Today's Agenda: *CNNs and Sequence Data*

Today, we go over some known deep

Deep Convolutional NNs

We then start our final journey which explores briefly

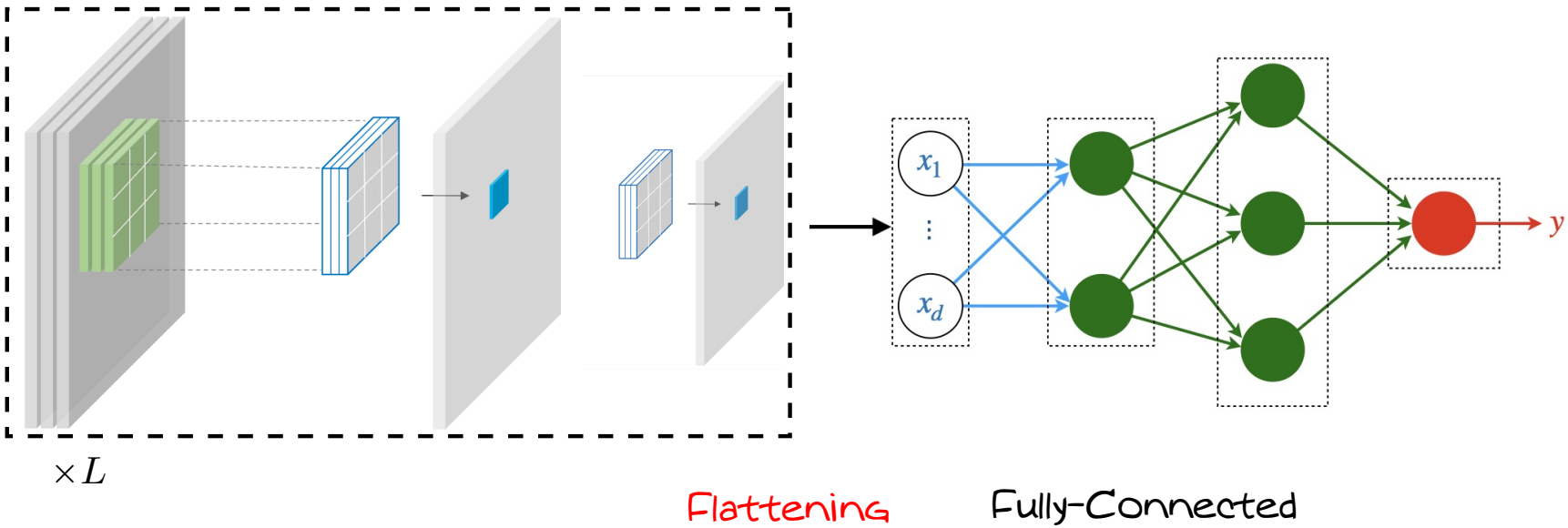
Advanced Topics in Machine Learning

In today's episode, we talk about

- *Sequence Data and Basic Sequence Models*

CNN: General Architecture

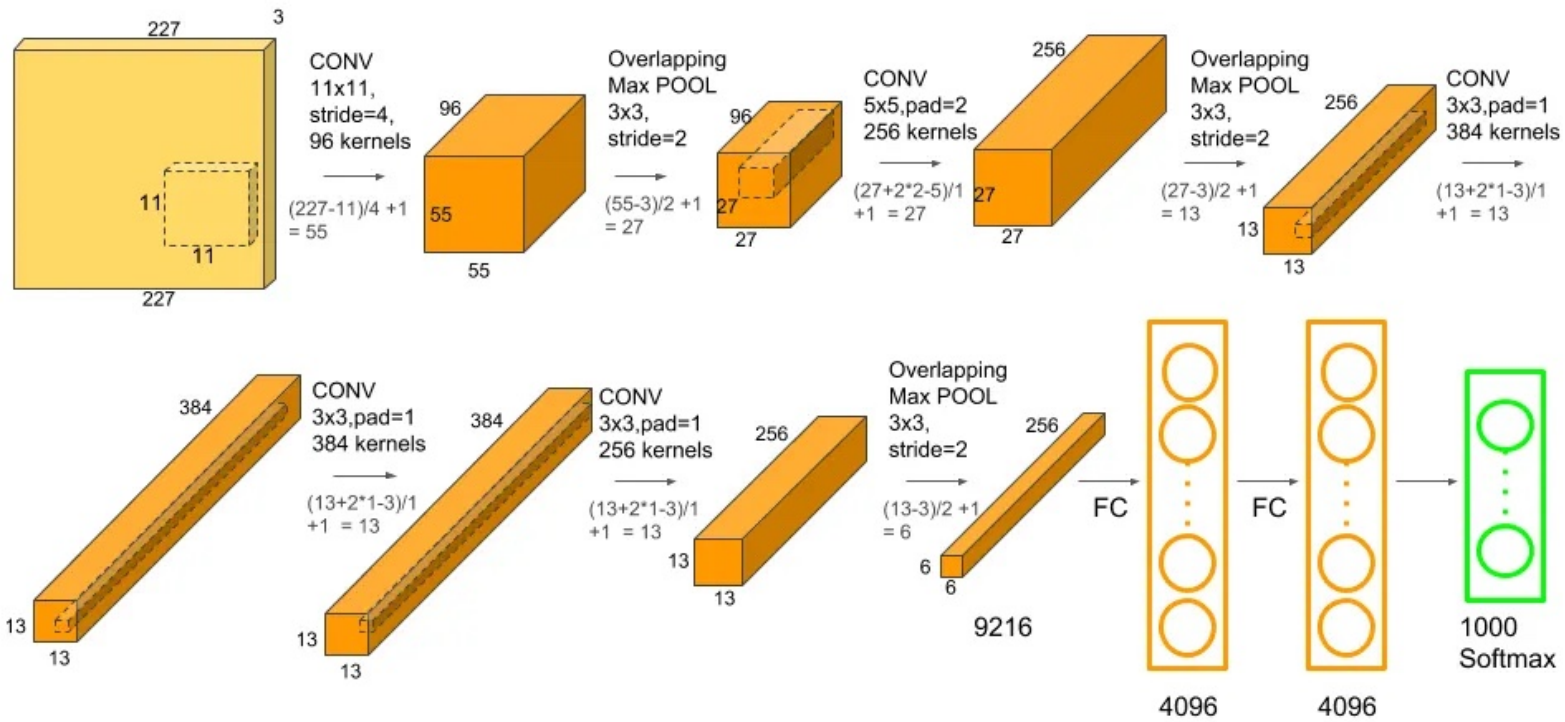
A CNN consists of multiple convolutional units and a fully-connected NN



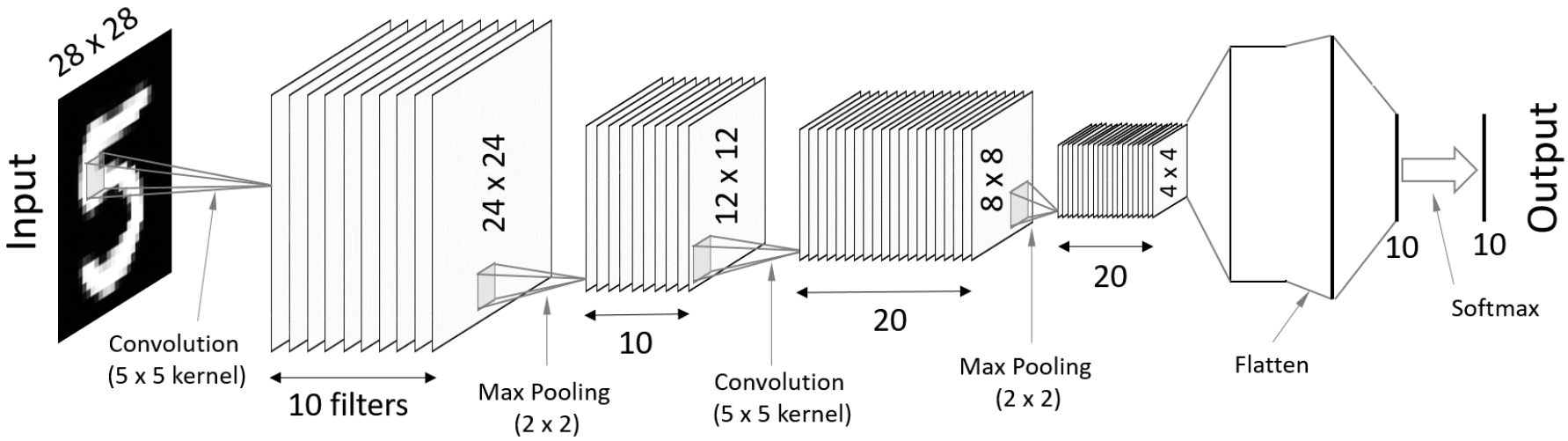
We can repeat the convolutional unit multiple times

Example: *AlexNet*

Let's look at the winner of the ImagNet challenge in 2012



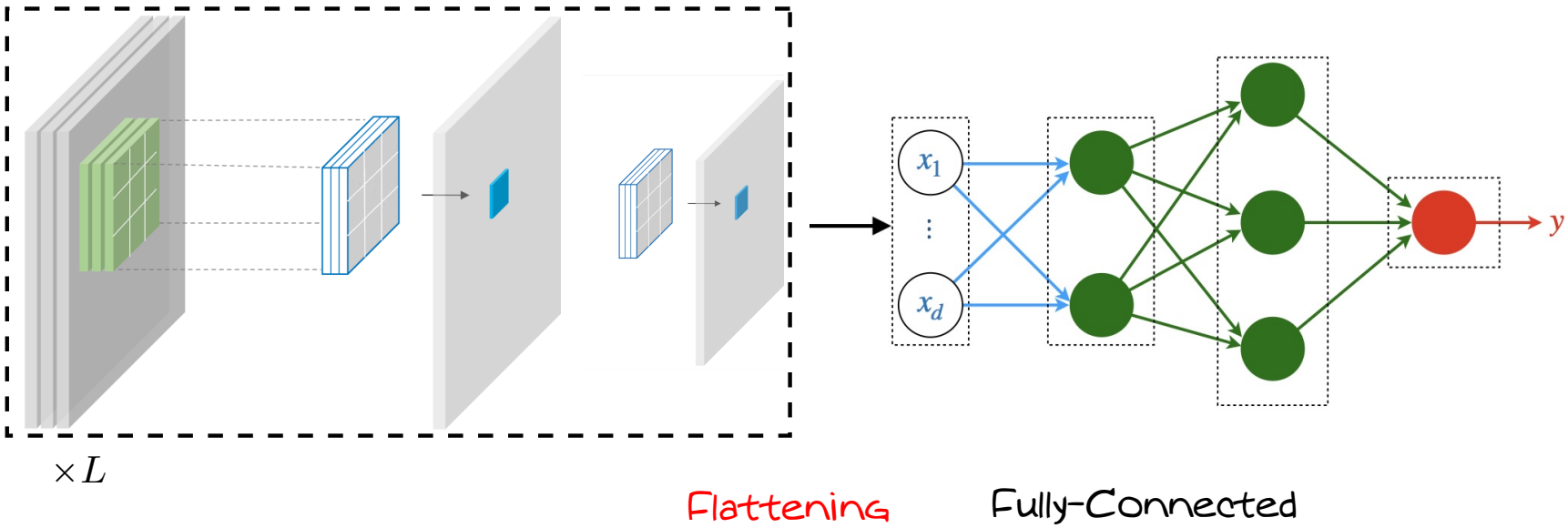
Example: *Custom CNN for MNIST Classification*



In this network, we do the following

- *We apply convolution with 10 filters*
  - ↳ *We apply pooling with stride 2*
- *We apply convolution with 20 filters*
  - ↳ *We apply pooling with stride 2*
- *We flatten and pass through one fully-connected layer*
  - ↳ *We compute Softmax for classification*

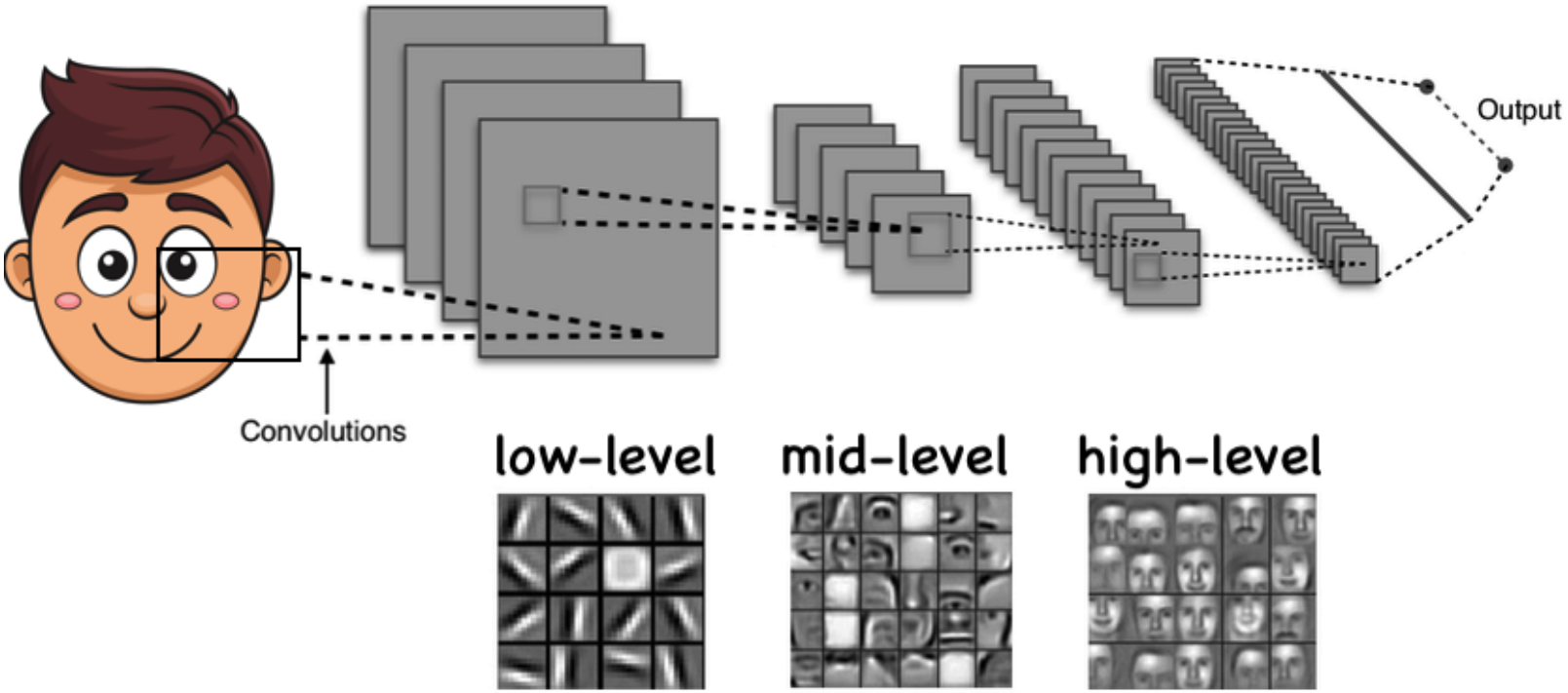
Backpropagation



It is easy to see that backward pass is dual to the forward pass

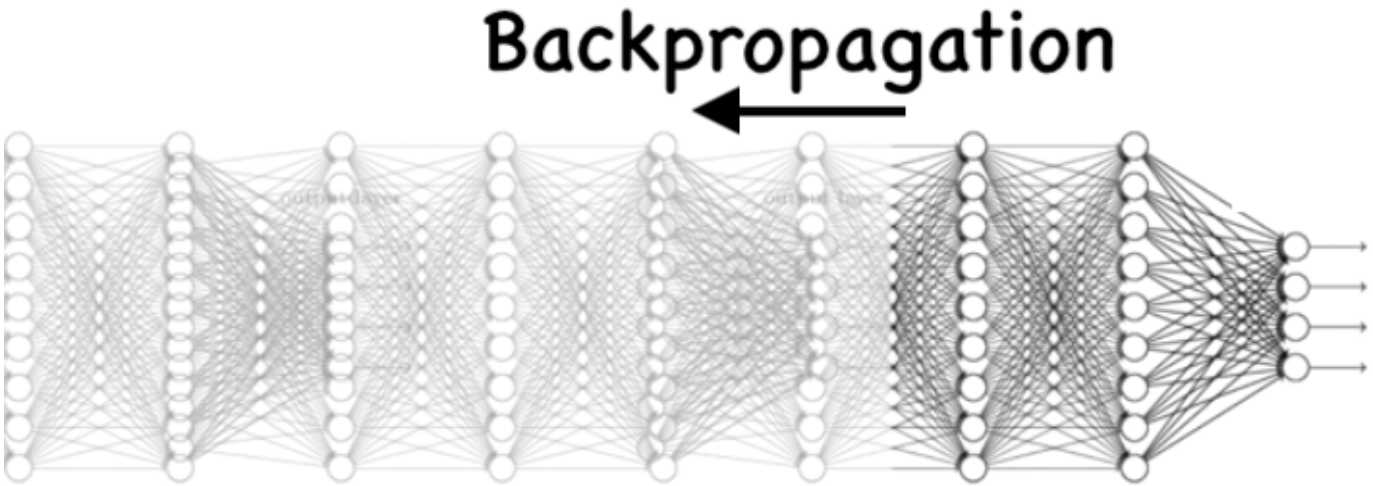
- *We can compute sample gradients with backpropagation*
- *Using mini-batch SGD, we can efficiently train CNNs*

Gradual Feature Extraction



As we go deeper, CNN extracts higher levels of features

Vanishing or Exploding Gradient

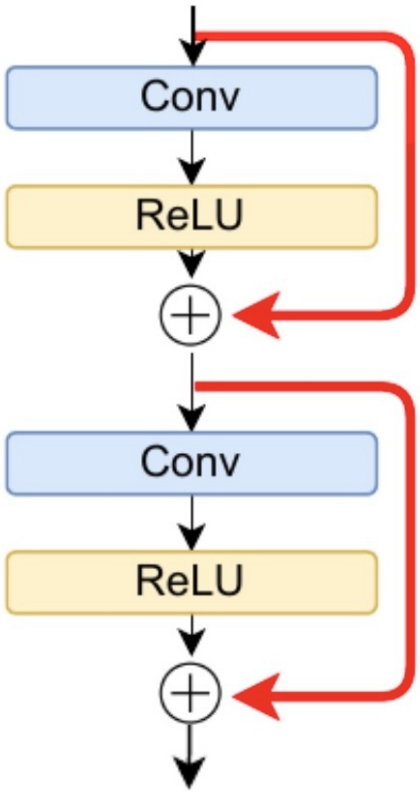


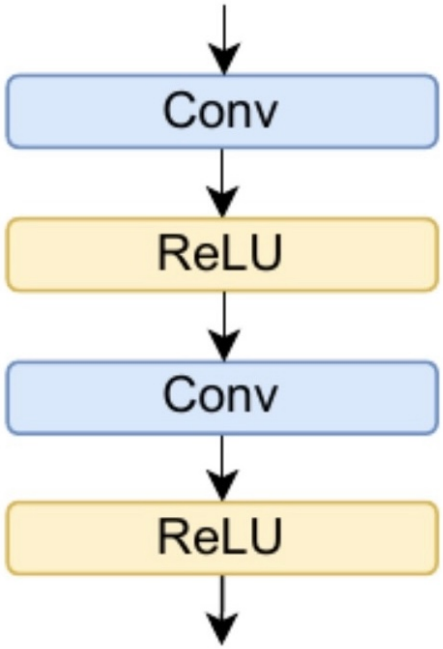
As we go very deep in FNNs, we could experience

- *Decrease in gradient values through depth*
  - ↳ *This results in vanishing gradient*
- *Increase in gradient values through depth*
  - ↳ *This results in exploding gradient*

ResNet: *Skip Connection*

ResNet uses skip connections to overcome this issue





Further Read

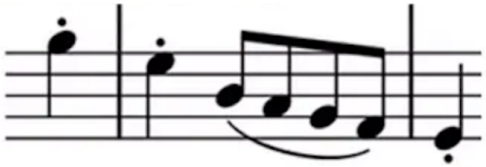
- Goodfellow
  - ↳ Chapter 9

CNNs

CNNs are further discussed in details in

- *ECE1508: Applied Deep Learning*
  - ↳ *Given in both Fall and Winter Semesters*

Sequence Data: *Many Applications*



"This product is useless!"

"This is the first lecture on ..."

Classic/Jazz/Pop/Hip-hop

A cute puppy in snow

Sport/Drama/Documentary

Sequence Learning Problem

Basic FNNs cannot be used in practice: *we need a huge input and/or output*

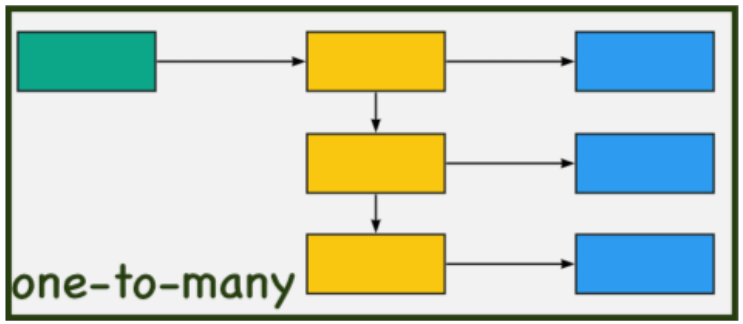
Sequence Learning Model

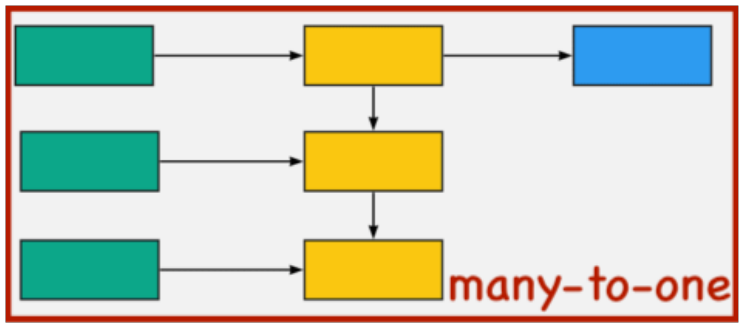
Models that get sequence inputs and return sequence outputs

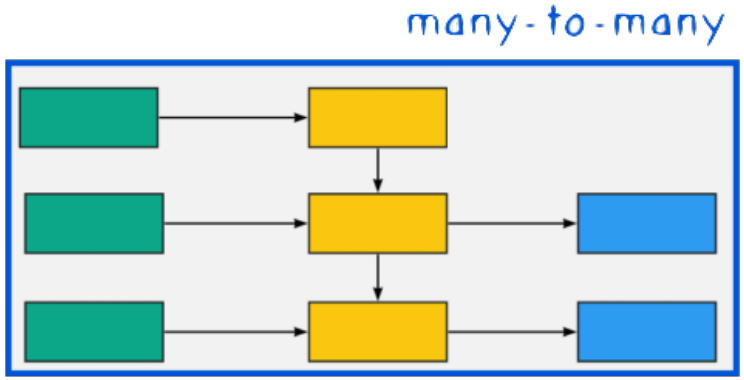
There are various approaches in the literature

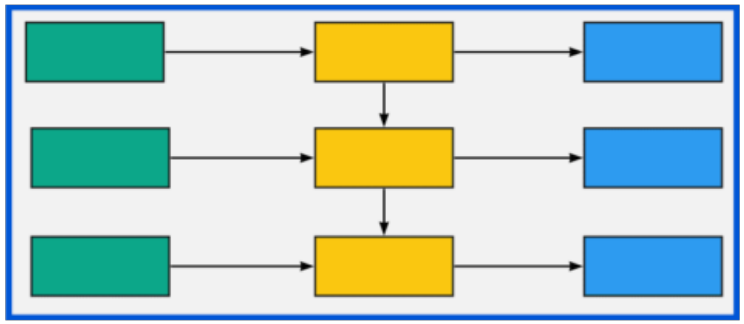
- *Recurrent Neural Networks (RNNs)*
- *Encoder-Decoder Architecture (Seq2Seq Models)*
- *Transformers*

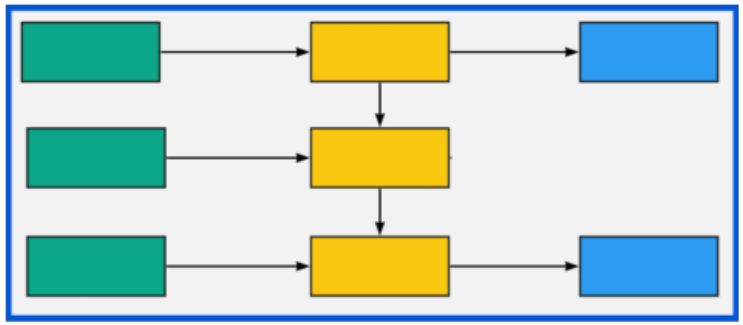
Types of Sequence Problems









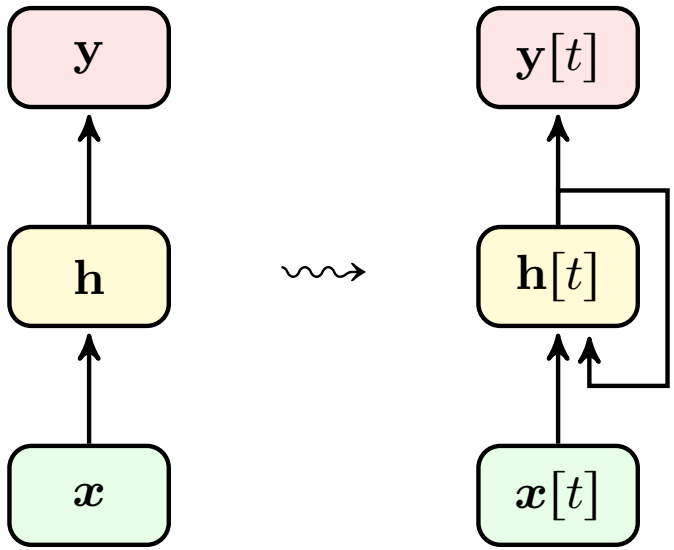


Recurrent Neural Networks

We can feed a sequence to a neural network, if we include recurrence

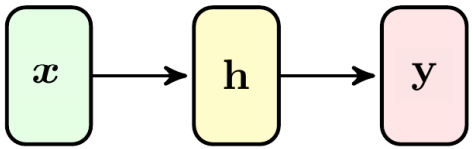
Recurrence

NN takes a state as input and returns the next state as output



Example: *Turning Shallow FNN to Basic RNN*

Say we have a shallow fully-connected FNN



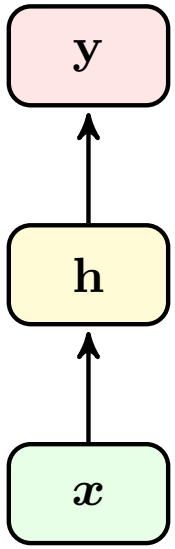
- *It computes hidden feature from the input* $\boldsymbol{x}$

$$\mathbf{h} = f\left(\mathbf{W}_1 \boldsymbol{x}\right)$$

- *It computes the output from the hidden features*

$$\mathbf{y} = f\left(\mathbf{W}_2 \mathbf{h}\right)$$

Example: *Turning Shallow FNN to Basic RNN*



The model in this case gives us

$$\mathbf{y} \propto P\left(v|\boldsymbol{x}\right)$$

and when we train it, we maximize its likelihood

Example: *Turning Shallow FNN to Basic RNN*

We can make an RNN by using the previous feature in each time: at time t

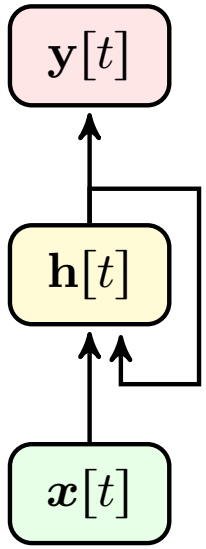
- *compute new features from the input $\boldsymbol{x}[t]$ and previous features $\mathbf{h}[t-1]$*

$$\mathbf{h}[t] = f\left(\mathbf{W}_1\boldsymbol{x}[t] + \mathbf{W}_0\mathbf{h}[t-1]\right)$$

- *computes the output from the new features*

$$\mathbf{y}[t] = f\left(\mathbf{W}_2\mathbf{h}[t]\right)$$

Example: *Turning Shallow FNN to Basic RNN*



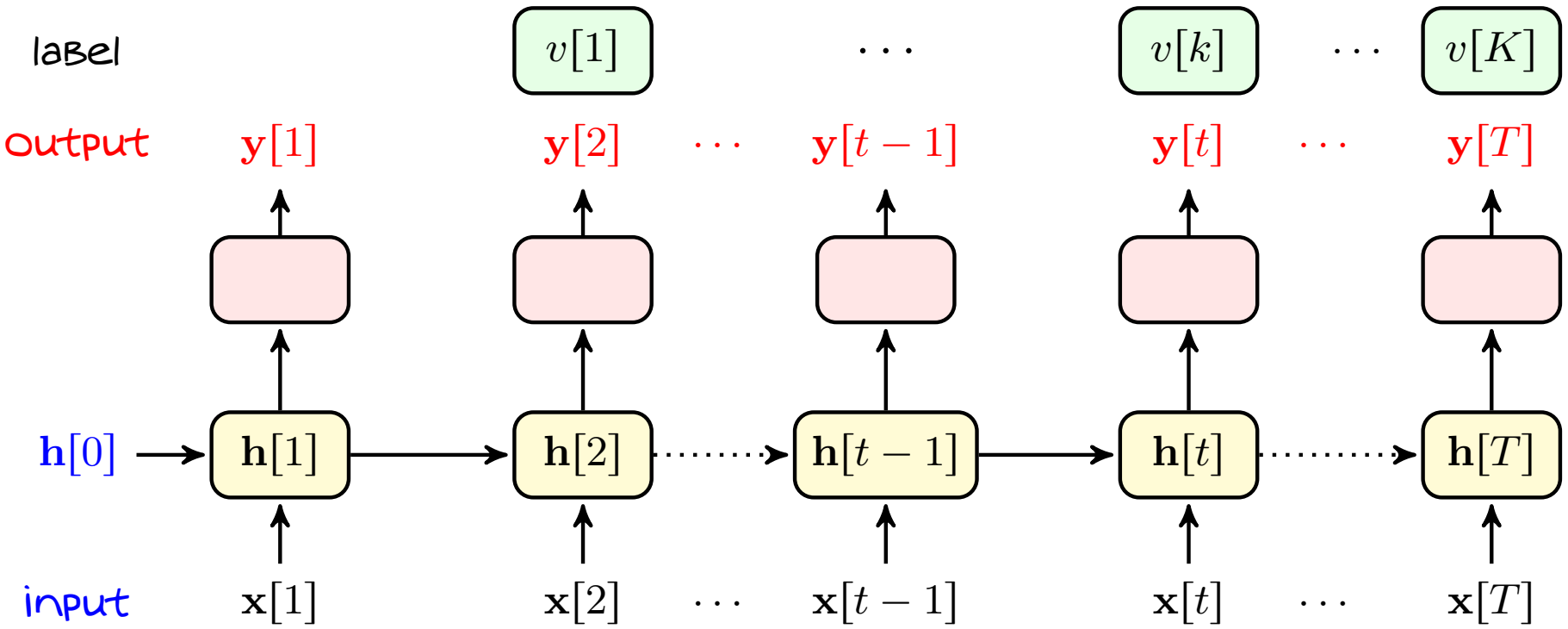
This recursive model determines for us

$$\mathbf{y}[t] \propto P\left(v[t] | \mathbf{h}[t-1], \boldsymbol{x}[t]\right) \equiv P\left(v[t] | \boldsymbol{x}[1], \ldots, \boldsymbol{x}[t]\right)$$

and we should maximize the likelihood over time

↳ *information about* $\boldsymbol{x}[1], \ldots, \boldsymbol{x}[t-1]$ *is somehow encoded in* $\mathbf{h}[t-1]$

Example: *Turning Shallow FNN to Basic RNN*



Main Challenge: *Vanishing Gradient Through Time*

When we train an RNN, we should backpropagate through time

backpropagation through time $\approx$ *backpropagation through deep NNs*

- *Exploding Gradients*
- *Vanishing Gradients*
  - ↳ *Gradients become small over time*
  - ↳ *Weights of RNN are updated only with most recent time instances*
  - ↳ *RNN has a limited memory!*

Classical Remedies

There are various approaches to handle these issues

- *Clipping gradients when exploding*
  - ↳ *Usually used to deal with exploding gradient*
- *Truncated backpropagation through time*
  - ↳ *Updated multiple times in the middle to carry memory forward*
- *Using gated units*
  - ↳ *Use the so-called gate to control better the memory*

They however work to some extent

- *We go in practice to more sophisticated architectures*
  - ↳ *The most well-known is the transformer*

Further Read

- Goodfellow
  - ↳ Chapter 10 ***RNNs***

RNNs and Transformers are discussed in

- *ECE1508: Applied Deep Learning*
  - ↳ *Given in both Fall and Winter Semesters*
- *ECE1786: Creative Applications of NLP*
  - ↳ *Given in Fall Semesters*